

Del III: Skiskyting og datastrukturer (180p)

Del 3 av eksamen er programmeringsoppgåver. Denne delen inneheld **11 oppgåver** som til saman gir ein maksimal poengsum på 180 poeng og tel **ca. 60 %** av eksamen. Kvar oppgåve gir maksimal poengsum på **5 - 25 poeng** avhengig av vanskegrad og arbeidsmengd.

Den utdelte koden inneheld kompilerbare (og kjørbare) .cpp- og .h-filer med forhåndskoda delar og ei full beskrivelse av oppgåvene i del 3 som ei PDF. Etter å ha lasta ned koden står du fritt til å bruke eit utveklingsmiljø (VS Code) for å jobbe med oppgåvene.

Du kan lasta ned .zip-fila med den utdelte koden frå Inspira. I neste seksjon finn du ei beskriving av korleis du gjer dette. Før du leverer eksamen, må du hugse å lasta opp ei .zip-fil med den utdelte koden og endringene du har gjort når du har løyst oppgåvene.

Nedlasting og oppsett av den utdelte koden

Dei fem stega under beskriv korleis du kan lasta ned og setje opp den utdelte koden. Ei bildebeskriving av stega kan du sjå i Figur 1g.

1. Last ned .zip-fila frå Inspira.
2. Pakk ut .zip-fila ved å trykje på Extract.
3. I vindauget som kjem opp skriv du «C:\temp».
4. Opne mappa som ligg i C:\temp i VS Code ved å velje:

File → Open Folder ...

5. Køyr følgjande kommando i VS Code:

`Ctrl+Shift+P → TDT4102: Create project from TDT4102 template → Configuration only`

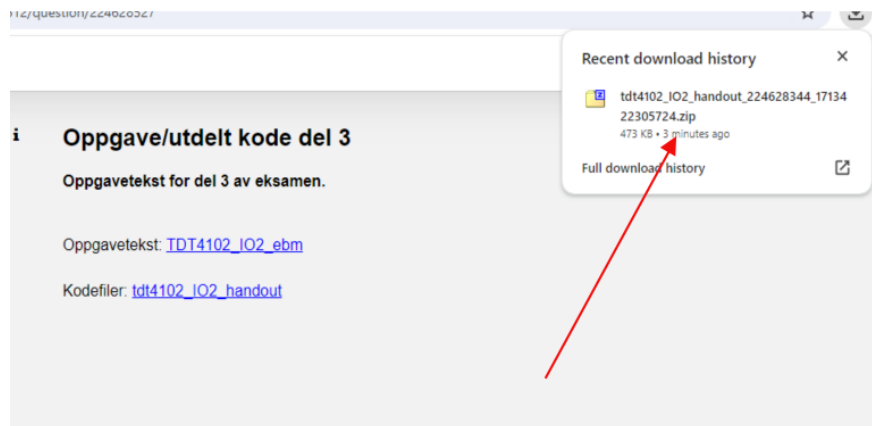
Sjekkliste ved tekniske problem

Viss prosjektet du opprettar med den utdelte koden ikkje kompilerer (før du har lagt til eigne endringer) bør du rekkje opp hânda og be om hjelp. Medan du venter kan du prôva følgjande:

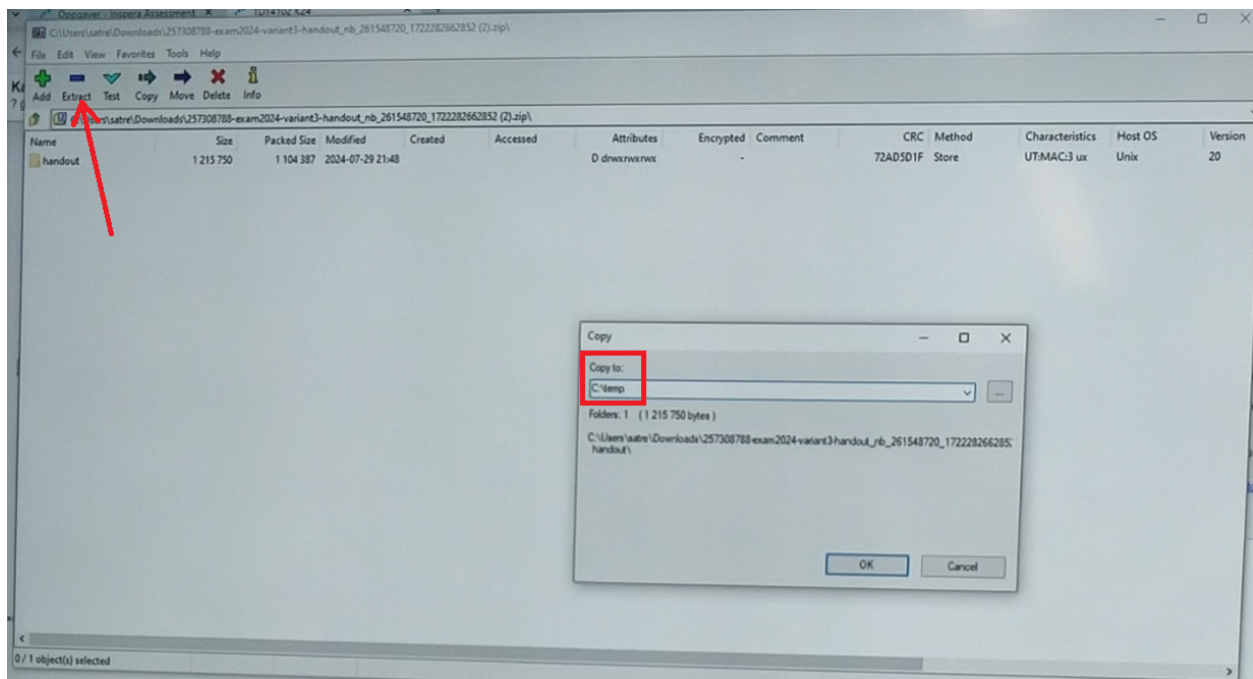
1. Sjekk at prosjektmappa er lagret i mappa C:\temp.
2. Sjekk at namnet på mappa **IKKJE** inneheld norske bokstavar (*Æ, Ø, Å*) eller mellomrom. Dette kan skape problem når du prøver å køyre koden i VS Code.
3. Sørg for at du er inne i rett mappe i VS Code.
4. Køyr følgjande TDT4102-kommando:
(a) `Ctrl+Shift+P → TDT4102: Create project from TDT4102 template → Configuration only`
5. Prøv å køyra koden igjen (F5, Ctrl+F5, eller Fn+F5).
6. Viss det framleis ikkje fungerer, lukk VS Code vindauget og opne det igjen. Gjenta deretter steg 5-6.



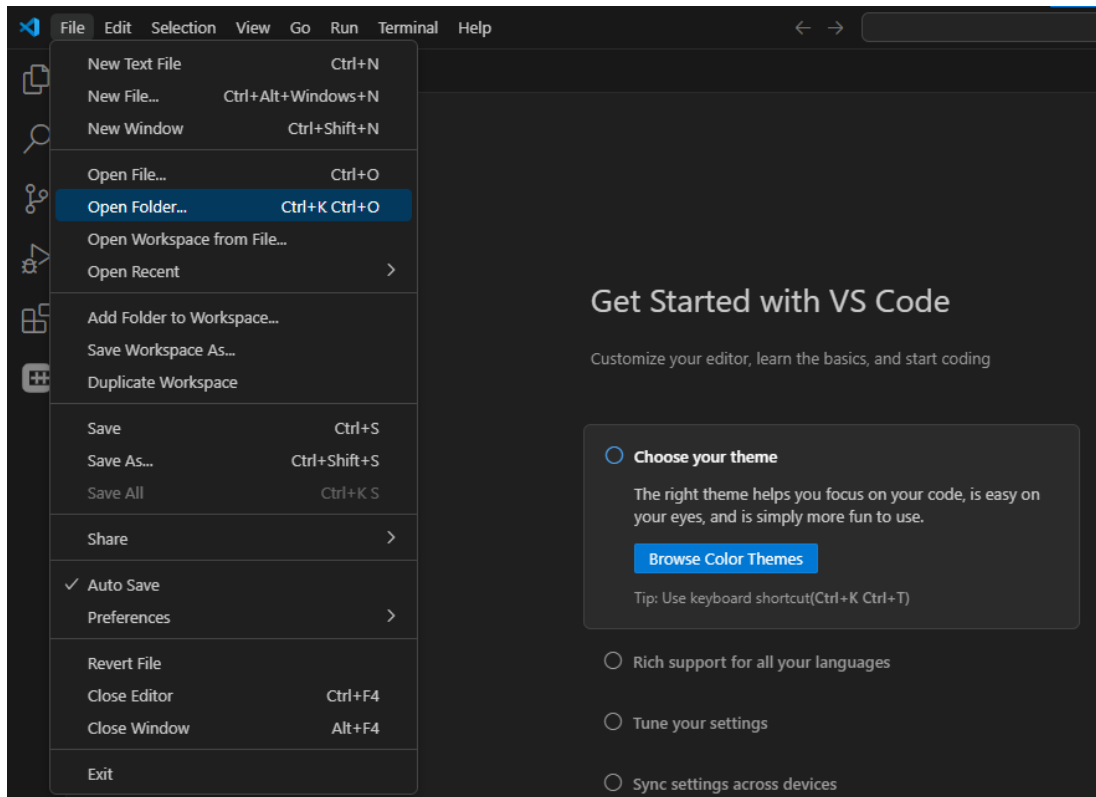
(a) Steg 1



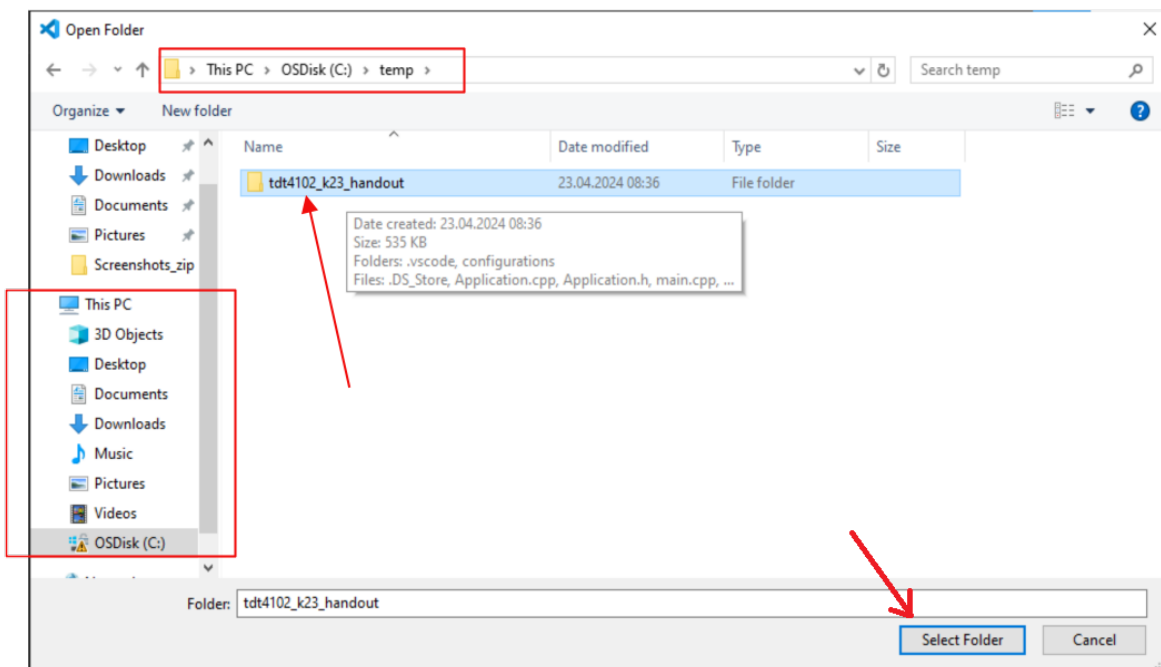
(b) Steg 1



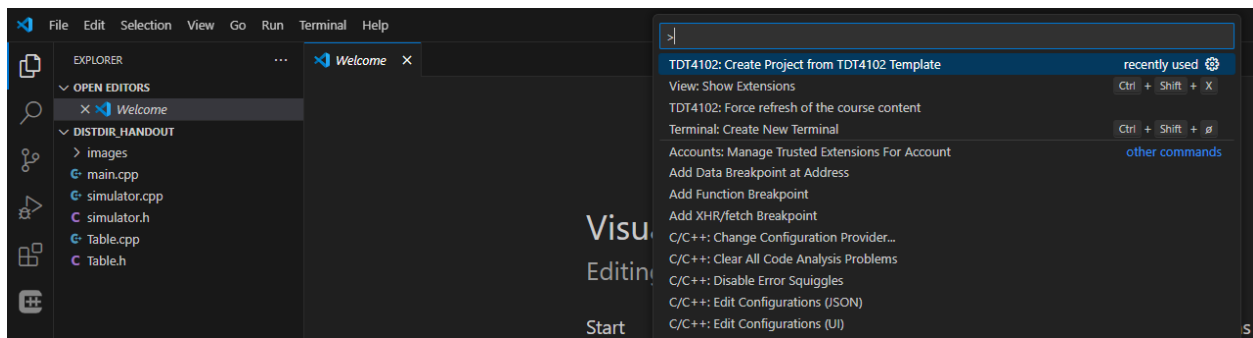
(c) Steg 2 og 3: Skriv inn «C:\temp» i vindauget og klikk OK.



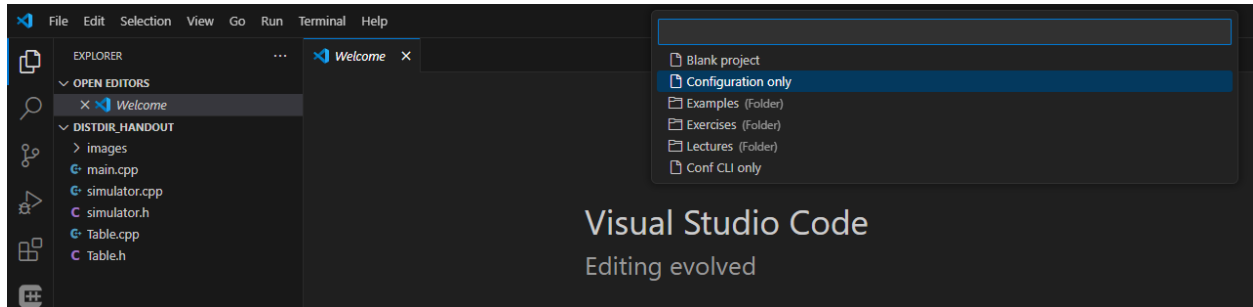
(d) Steg 4



(e) Steg 4



(f) Steg 5



(g) Steg 5

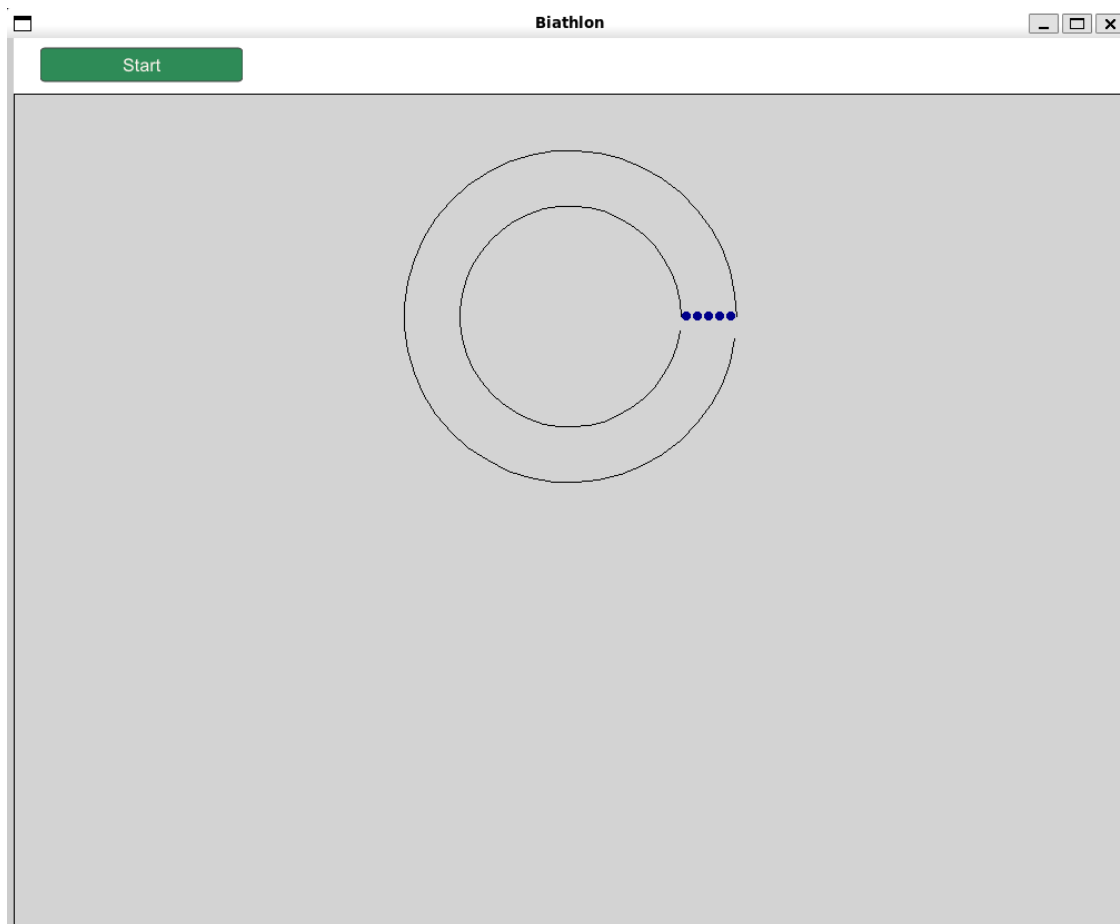
Introduksjon

Del 3 er delt inn i to hovudbolkar:

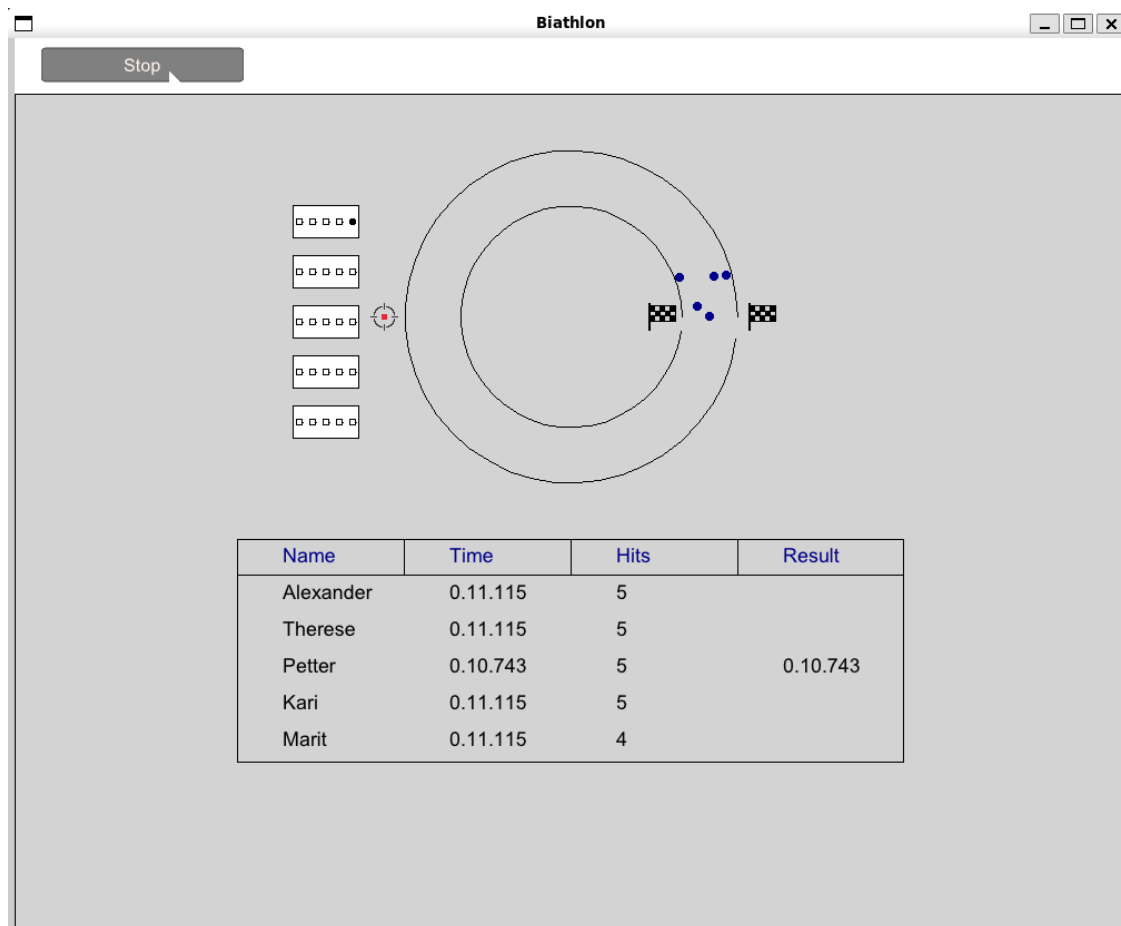
- Ei skiskytter-simulering som produserer data.
- Ein tabellstruktur (tenk rekneark som i Excel) som viser fram dataen og held oversikt over resultata.

Figur 2 viser korleis det ser ut når den utdelte koden blir køyrt. Figur 3 viser korleis det ser ut når koden blir køyrt etter at alle oppgåvene er løyst.

Simulasjonen blir kontrollert av eit Simulator-objekt som har ei liste med alle Skier-objekta. I tillegg har Simulator ein `unique_ptr` til eit Stopwatch-objekt som registrerer tida, og eit Table-objekt for resultata.



Figur 2: Skjermbilete av korleis det ser ut når den utdelte koden blir køyrt. I animasjonsvindaugget kan ein sjå ein funksjonsbar beståande av ein Start/Stopp-knapp og to sirklar som skal illustrera ei skiløype.



Figur 3: Skjermbilete av korleis det ser ut når koden blir køyrt etter at alle oppgåvane er løyst.

Tabellen er generell, slik at kvar celle i teorien kan inneholde virkårleg datatype, men tabellen er initialisert med `std::string` i simulasjonen. Eit `Table`-objekt lagrar verdiane i ei to-dimensjonal liste kalla `data`. Kvar celle i tabellen består av ein `TableData`-struktur. For å holda kontroll på kva for nokon kolonner som finst i tabellen, blir i tillegg kolonnenamna lagra i lista `columnTitles`. Denne lista inneheld alle dei ulike strengane ein kan finne i felte `TableData::columnTitle` i `data`. Under kan du sjå definisjonar for `TableData`-strukturen og delar av `Table`-klassen.

```
template <typename T>
struct TableData
{
    string columnTitle;
    T value;
};

class Table
{
public:
    void draw(AnimationWindow &window, int height, int width, Point topLeftCorner);
    void storeTable(filesystem::path filepath);
```

```

// (...)
// more functionality below for
// adding and updating values

private:
// (...)

vector<string> columnTitles;
vector<vector<TableData<string>>> data;
};

```

Fordelen med denne måten å lagre cellene på er at radene i data kun inneheld ei kolonne dersom den har ein verdi. Det betyr at dersom det blir lagt til ei ny kolonne, treng vi ikkje gå gjennom heile tabellen og leggje til NULL-verdiar for radene som allereie eksisterer. Sagt på ein annan måte, lagrer vi ikkje NULL-verdiar viss dei ikkje eksplisitt blir lagt inn.

I Figur 4, Figur 5 og Figur 6 kan du sjå eit eksempel på korleis dette kan henge saman.

	A	B	C	D
1	Navn	Studie	Retning	Bosted
2	Henrik	Sykepleie		Bergen
3	Ola			Trondheim
4	Kari	Psykologi		
5	Anne	Veterinær	Smådyr	Ås

Figur 4: Eksempel på ein tilfeldig tabell. Noter deg at ikkje alle feltene er fylt inn.

```

[0] [1] [2] [3]
{"Navn", "Studie", "Retning", "Bosted"}

```

Figur 5: Korleis lista columnTitles kan sjå ut for tabellen i Figur 4. Tala over lista visar indeksen til dei ulike verdiane.

```

{
  [0] [1] [2] [3]
  {"Navn": "Henrik", "Studie": "Sykepleie", "Bosted": "Bergen"},
  {"Navn": "Ola", "Bosted": "Trondheim"},
  {"Navn": "Kari", "Studie": "Psykologi"},
  {"Navn": "Kari", "Studie": "Veterinær", "Retning": "Smådyr", "Bosted": "Ås"}
}

```

Figur 6: Korleis den 2-dimensjonale lista data kan sjå ut for tabellen i Figur 4. Tala over lista skal visa indeksen til dei ulike verdiane. Noter deg at ikkje alle vektorane er like lange. Noter deg også at alle radene er sortert i same rekkefølgd som columnTitles. Det vil seie at 'Navn' alltid kjem først og 'Bosted' alltid kjem sist.

Din oppgåve er å utvide og leggje til funksjonalitet i den utdelte koden. I .zip-fila finn du eit kodeskjelett med tydeleg markerte oppgåver.

Korleis svare på oppgåvene

Dersom du synast nokon av oppgåvene er uklare kan du oppgje korleis du tolkar dei og det du antar for å løyse oppgåva som kommentarar i koden du leverer.

Oppgåveteksten

Oppgåvetekstene vil ha følgjande struktur:

Første del inneheld motivasjon, bakgrunnsinformasjon og korleis koden er satt saman for oppgåven.

Neste del er ein tekstboks som inneheld oppgåven og spesifikke krav.

Siste del forklarar resultatet du kan forvente når du har fullført oppgåven.

Koden

Kvar oppgåve i del 3 har ein unik kode for å gjere det lettare for deg å vite kor du skal fylle inn svara dine. Koden er på formatet `<T><siffer>` (TS), der siffera er mellom 1 og 11 (*T1* - *T11*). For kvar oppgåve vil ein finne to kommentarar som skal definere høvesvis begynninga og slutten av svaret ditt. Kommentarene er på formatet: `// BEGIN: TS` og `// END: TS`.

For eksempel kan ei oppgåve sjå slik ut:

```
// TASK T1
bool foo(int x, int y) {
// BEGIN: T1
// Write your answer to assignment T1 here, between the // BEGIN: T1
// and // END: T1 comments. You should remove any code that is
// already there and replace it with your own.

return false;

// END: T1
}
```

Etter at du har implementert løysinga di bør du enda opp med følgjande:

```
// TASK T1
bool foo(int x, int y) {
// BEGIN: T1
// Write your answer to assignment T1 here, between the // BEGIN: T1
// and // END: T1 comments. You should remove any code that is
// already there and replace it with your own.
    return (x + y) % 2 == 0;
// END: T1
}
```

Det er veldig viktig at alle svara dine blir ført mellom slike par av kommentarar. Dette er for å støtte sensurmekanikken vår. Viss det allereie er skriven kode *mellom* BEGIN- og END-kommentarane til ei oppgåve i kodeskjelettet som er gitt ut, så kan du, og ofte bør du, erstatte denne koden med din eigen implementasjon. All kode som er skriven *utanfor* BEGIN- og END-kommentarane **SKAL** du la stå som den er.

Merk: Du skal **IKKJE** fjerne BEGIN- og END-kommentarane.

Tips: trykker ein CTRL+SHIFT+F og søker på BEGIN: får ein snarvegar til starten av alle oppgåvene lista opp i utforskervindauget så ein enkelt kan hoppe mellom oppgåvene. For å kome tilbake til det vanlege utforskervindauget kan ein trykke CTRL+SHIFT+E.

Korleis levere del 3

Når du er ferdig med oppgåvene og er klar til å levere skal du laste opp alle .h- og .cpp-filene i hovudmappa som ei .zip-fil i Inspira. Du står fritt til å bruke den innebygde funksjonen i VS Code til å lage .zip-fila. Desse filene inkluderer:

- Table.h
- Table.cpp
- Simulator.h
- Simulator.cpp
- main.cpp

Oppgåvene

1. (5 points) T1: Start simulasjonen

Start-knappen treng tilgang til variablane `running` og `watch` i `Simulator`-klassen for å kunne køyre simulasjonen. Vi ynskjer derimot ikkje å gjere variablane offentlege, verken ved å gjere dei `public` eller gjennom get-funksjonar. Din oppgåve er å finne ei anna måte å gi `ToggleButton`-klassen tilgang til desse variablane.

I `simulator.h` filen

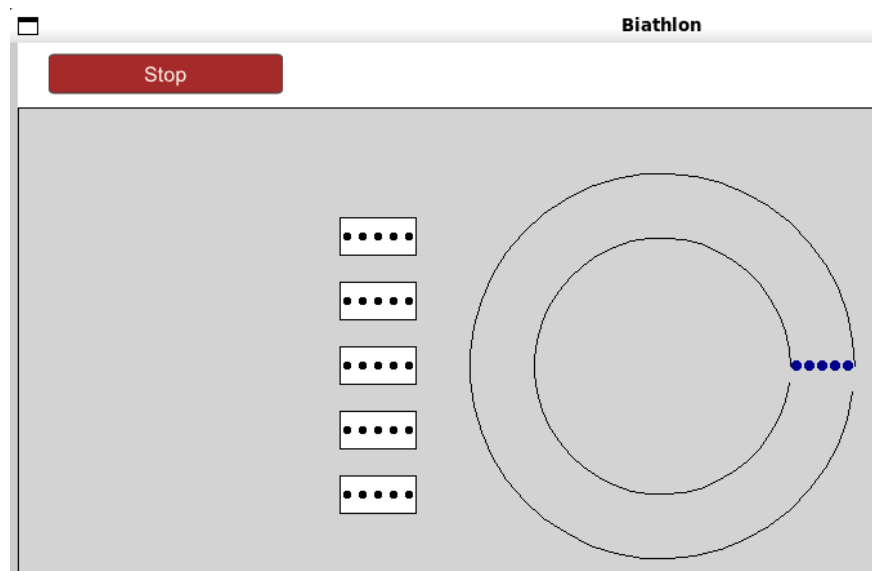
- Fjern eller kommenter ut kodelinja
`#define START_BUTTON_NO_ACCESS_TO_RUNNING_AND_WATCH`
i `simulator.h`.
- Gi `ToggleButton`-klassen tilgang til variablane `running` og `watch` utan å endre tilgangsnivået frå `private`.

Tips 1: Du kan leggja til kodelinja `#define START_BUTTON_NO_ACCESS_TO_RUNNING_AND_WATCH` igjen dersom du ikkje har løyst oppgåva enda og ynskjer å fjerne feilmeldingene du får frå kompilatoren.

Tips 2: Dersom du ikkje får til denne oppgåva kan du kommentere ut eller fjerna linje 155 i `simulator.h`. Dette vil gje alt i `Simulator` tilgangsnivået `public`, og du kan fortsette på dei neste oppgåvene utan ulemper.

```
// Remove this part to make everything public if you
// did not manage to complete task T1
// REMOVE
private:
// REMOVE
```

Når oppgåve T1 er ferdig skal simuleringa starte når ein trykker på Start-knappen. Dette ser ein ved at knappen byter mellom Start-knapp og Stop-knapp når den blir trykt på. I tillegg vil ein etter ca. 5 sekunder sjå at målskivene til utøverane visast når de ankommer skytebanen som vist i Figur 7.



Figur 7: Simulasjonen etter at T1 er implementert og ein har trykt på Start-knappen og venta nokon sekunder.

2. (5 points) T2: Oppdater antal treff

Når ein no trykkjer på Start-knappen vil simulasjonen køyre, men målskivene (sjå Figur 7) blir ikkje oppdatert når ein utøvar treffer. Det er fordi hit-funksjonen i Target ikkje er implementert. Denne funksjonen skal auke hits-variabelen i Target med 1 kvar gong den blir kalla. Du finn definisjonen til Target-klassen i `simulator.h`, som ser slik ut:

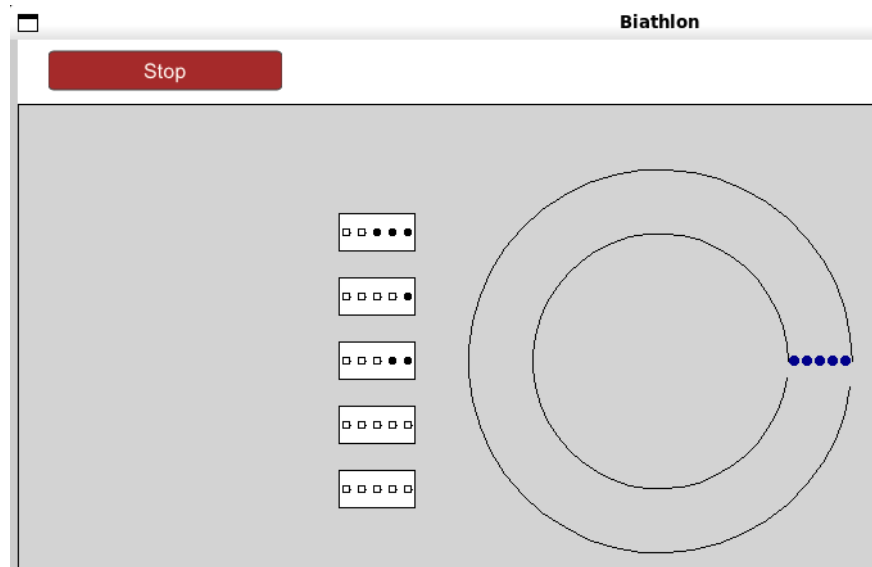
```
class Target
{
public:
    void hit();
    void draw(AnimationWindow *window, Point position, int size);
    int getHits();

private:
    int hits = 0;
};
```

Implementer funksjonen `Target::hit()` i `simulator.cpp`.

- `Target::hit()` auke `Target::hits` variabelen med 1.

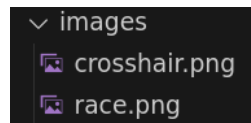
Når oppgave T2 er implementert vil ein kunne sjå at målskivene blir oppdatert etter kvart som utøvarane skyt og treffer som vist i Figur 8.



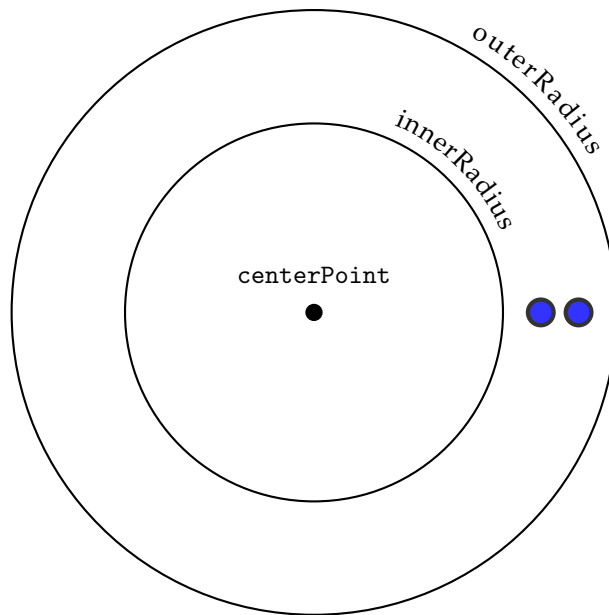
Figur 8: Målskivene vil bli oppdatert når ein utøvar treffer etter at oppgåve T2 er implementert.

3. (10 points) T3: Marker kartet

Vi ynskjer å markere start, mål og skyteområde på kartet i simulasjonen. I `images`-mappa i den utdelte koden finn du to bilete (`.png`-filer). Fila `crosshair.png` viser eit trådkors og fila `race.png` viser eit flagg.



Vi skal no implementere funksjonen `Simulator::placeImages` som skal bruke bileta til å markere kartet. Denne funksjonen tek 4 parametrar: `centerPoint`, `innerRadius`, `outerRadius` og `size.size` beskriv storleiken som skal brukast på bileta. Dei andre variablane er illustrert i Figur 9, og skal brukast til å plassere bileta på riktig stad.



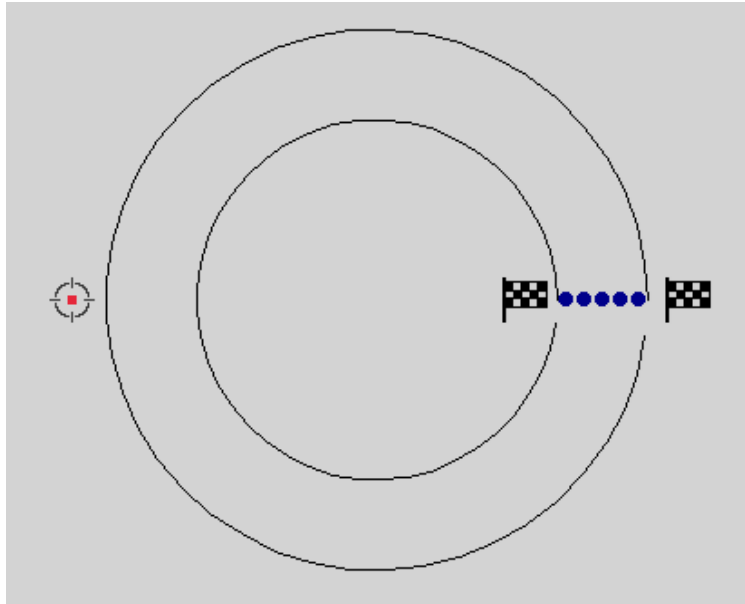
Figur 9: `centerPoint` er midtpunktet vi orienterer fra. `innerRadius` er avstanden fra `centerPoint` til den innerste sirkelen. `outerRadius` er avstanden fra `centerPoint` til den yste sirkelen.

Implementer funksjonen `Simulator::placeImages` i `simulator.cpp`.

- Bruk parametrane `centerPoint`, `innerRadius` og `outerRadius` til å teikne flagg ved start og mål, og eit trådkors ved skytområdet.
- Bildefilene (`.png`-filene) som skal brukast ligg i `images`-mappa. Fila `crosshair.png` viser eit trådkors og fila `race.png` viser eit flagg.
- `size`-parameteren skal brukast til å setje riktig storleik på bileta.

Hint: Ein kan finne informasjon om korleis ein brukar bilete i dokumentasjonen til `AnimationWindow`.

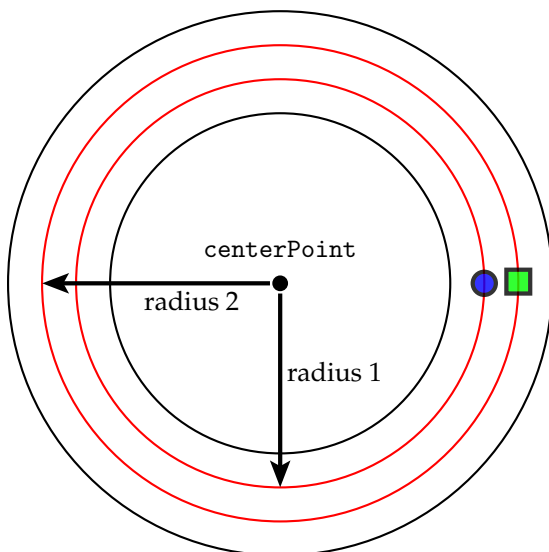
Når oppgave T3 er gjort skal kartet sjå ut omtrent som kartet i Figur 10. Det tek litt lenger tid før målskiva visast etter denne oppgåva er implementert.



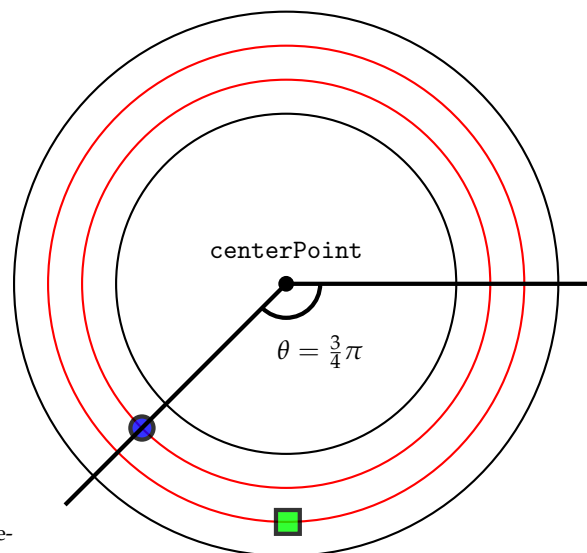
Figur 10: Skjermbilete av simulasjonen etter at oppgåve T3 er implementert, med flagg ved start og mål, og trådkors ved skyteområdet.

4. (20 points) T4: Få utøverane ut i løypa

Vi ynskjer å kunne visualisere progresjonen til kvar enkelt utøver. Siden kartet er gitt som ein sirkel, vil posisjonen til ein utøver vere gitt av ein *radius* som fortel kva for fil dei ligg og ein *vinkel* som fortel kor langt dei har kome. Dette er vist i høvesvis Figur 11a og Figur 11b.



(a) Radius til to utøvere, der *radius 1* er radiusen til utøveren representert med ein sirkel, og *radius 2* er radiusen til utøveren representert med ein firkant.



(b) Vinkelen til ein utøver på ei bane av lengde 1.

Figur 11: Oversikt over løypevariablar.

x -posisjonen til ein utøveren er dermed gitt av

$$x = x_c + r \cdot \cos \theta,$$

medan y -posisjonen til ein utøvar er gitt av

$$y = y_c + r \cdot \sin \theta,$$

kor x_c og y_c representerer `centerPoint` sin x - og y -koordinat, og r er radiusen. θ er progresjonen til utøveren og er gitt av

$$\theta = 2\pi \cdot \frac{\text{position}}{\text{distance}},$$

kor *distance* er lengda på banen og *position* er posisjonen til utøveren. Desse verdiane finn ein høvesvis i `Map::distance` og ved å bruka `Skier::getPosition()`. `Map`-strukturen er definert i `simulator.h` og er lagra som ein variabel i `Simulator`-klassen. `Skier::getPosition` er implementert i `simulator.cpp`.

```
struct Map
{
    double distance;
    double shootingPosition;
};

double Skier::getPosition() { return position; }
```

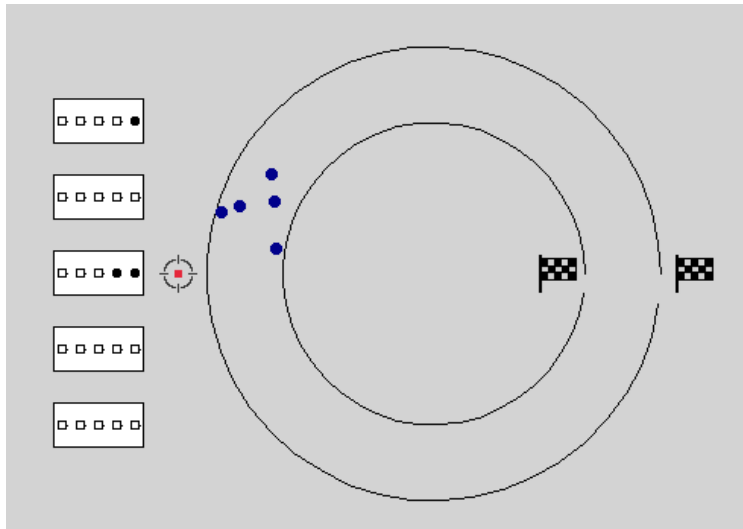
Implementer funksjonen `Simulator::calculateSkierPosition` i `simulator.cpp`.

- Funksjonen skal returnere eit `TDT4102::Point` som illustrerer ko i løypa utøveren er.
- x -posisjonen til ein utøvar er gitt av $x = x_c + r \cdot \cos \theta$.
- y -posisjonen til ein utøvar er gitt av $y = y_c + r \cdot \sin \theta$.

x_c og y_c representerer `centerPoint` sin x - og y -koordinat, og r er radiusen.

θ er progresjonen til utøveren og er gitt av $\theta = 2\pi \cdot \frac{\text{position}}{\text{distance}}$.

Når oppgåve T4 er implementert skal utøvarane bevege seg rundt på kartet under simuleringa som i Figur 12.



Figur 12: Skjermbilete av simuleringa etter at oppgåve T4 er implementert.

5. (20 points) T5: Fyll tabellen

Vi ynskjer ein tabell som viser korleis utøvarane ligger an i rennet. Denne tabellen er delvis implementert, men mangler mulighet for å legge til rader. Implementer template-funksjonen `Table::addRow` i `table.h`. Denne funksjonen skal ta inn ei rad i form av ein vector av `TableData` som parameter. `TableData`-strukturen er deklartert i `table.h` og ser slik ut:

```
1  template <typename T>
2  struct TableData
3  {
4      string columnName;
5      T value;
6  };
```

`Table`-klassen er definert i `table.h` og har medlemsvektorer `columnTitles` og `data`, kor høvesvis kolonnenamn og data er lagra:

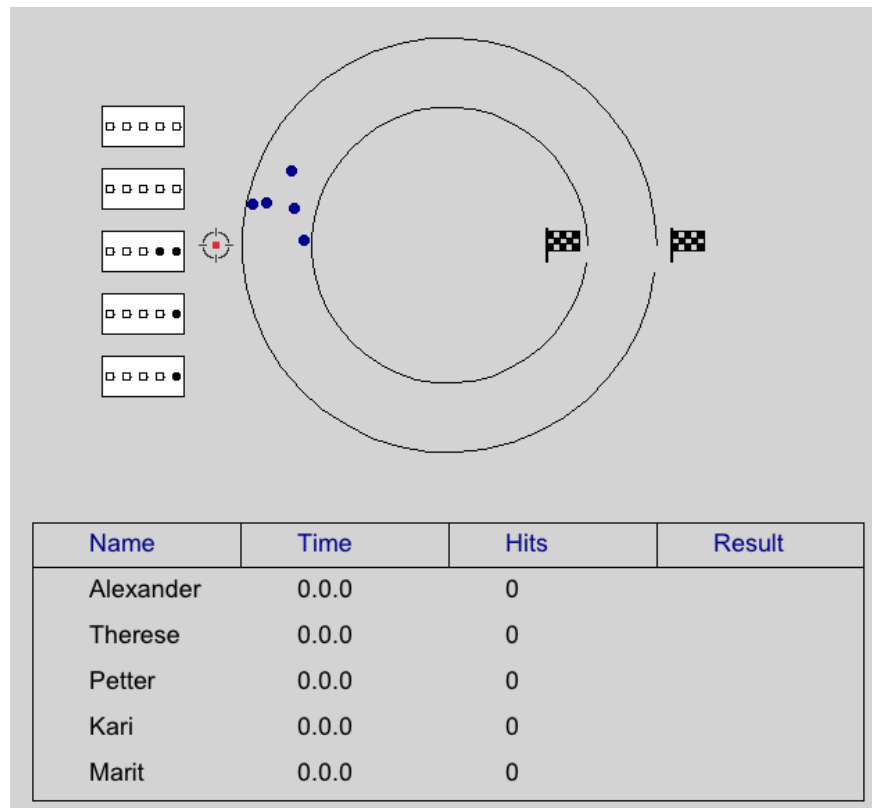
```
1  vector<string> columnTitles;
2  vector<vector<TableData<string>>> data;
```

Definer og implementer template-funksjonen `Table::addRow` i `table.h`. Funksjonen tek inn ein vector av `TableData` som parameter. Funksjonen har ingen returverdi. I tillegg:

- Fjern eller kommenter ut kodelinja `#define ADD_ROW_NOT_IMPLEMENTED` i `table.h`.
- Legg til kolonnenamnet i `Table::columnTitles`-vektoren viss det ikkje allereie eksisterer.
- Heile rada skal leggest til i `Table::data`-vektoren.

Tips: Du kan leggje til kodelinja `#define ADD_ROW_NOT_IMPLEMENTED` igjen dersom du ikkje har løyst oppgåva enda og ynskjer å fjerne feilmeldingene du får frå kompilatoren.

Når oppgåve T5 er gjort skal tabellen vere synleg i vindauget under simuleringa slik som i Figur 13.



Figur 13: Når oppgåve T5 er gjort skal tabellen vere synleg i vindauget under simuleringa.

6. (20 points) T6: Indeksering

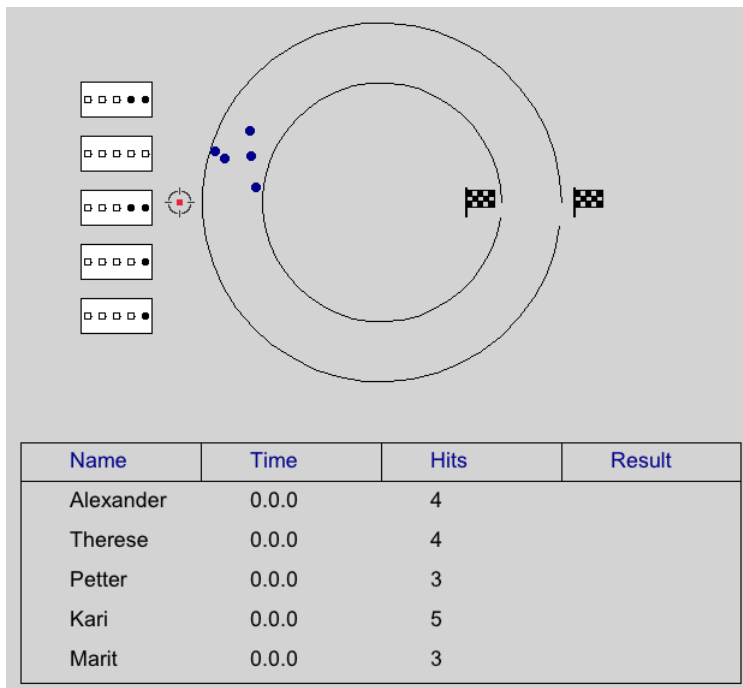
Indeksen til ei kolonne i tabellen samsvarar ikkje nødvendigvis med indeksen til kolonna i radene i data. For å finne ut kor dataen skal plasserast, må vi derfor kunne samanlikna `TableData::columnTitle` med kolonnenamna i `Table::columnTitles`.

Definer og implementer template-funksjonen `Table::getColumnIndexInRow` i `table.h`. Funksjonen skal ta inn ein `std::string` som beskriv kolonnetittel og ein vector av `TableData` som svarar til ei rad i data som parametrar. Funksjonen skal returnera ein `int` som svarar til indeksen til kolonn i rada. I tillegg

- Fjern eller kommenter ut kodelinja `#define GET_INDEX_NOT_IMPLEMENTED` i `table.h`.
- Returner indeksen til staden ein kan finne verdien til den gitte kolonnetittelen for denne spesifikke rada.
- Dersom kolonnetittelen vi ser etter ikkje finst i data skal funksjonen returnera -1.

Tips: Du kan leggje til kodelinja `#define GET_INDEX_NOT_IMPLEMENTED` igjen dersom du ikkje har løyst oppgåva enda og ynskjer å fjerne feilmeldingene du får frå kompilatoren.

Etter at oppgåve T6 er implementert vil du kunne sjå at tabellen oppdaterer verdiane i *Hits*-kolonna etter kvart som utøvarane treff blinken under skyting, som vist i Figur 14.



Figur 14: Skjermbilete av simulasjonen etter at oppgåve T6 er implementert.

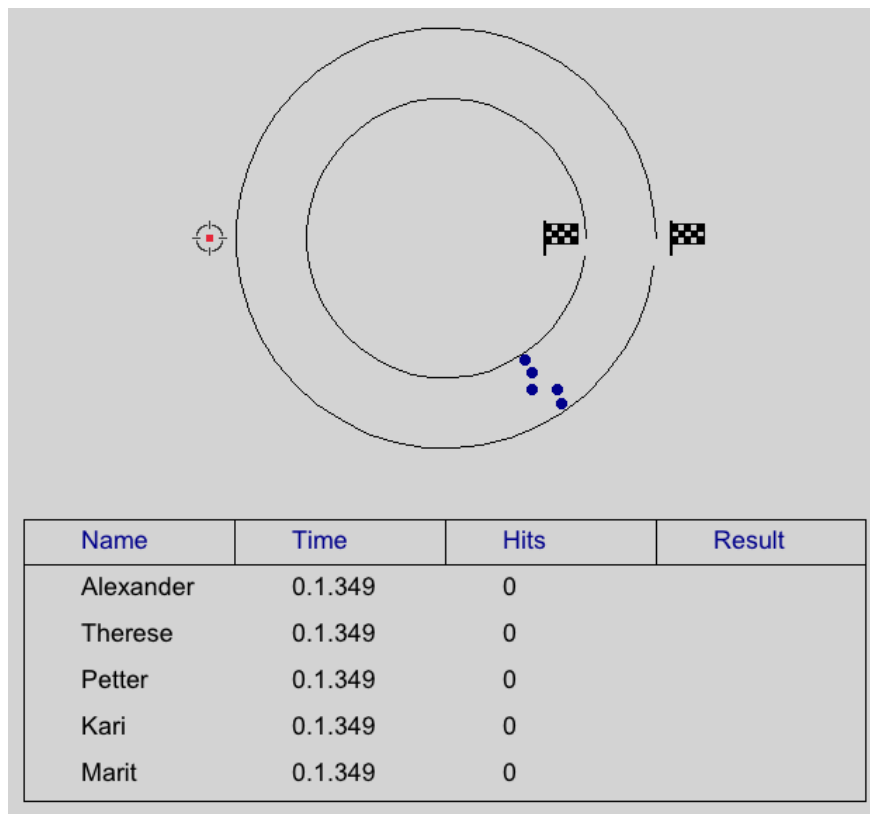
7. (15 points) T7: Kvart sekund tel!

Vi ynskjer å registrera tidene utøvarane brukar på løypa og vise det fram i tabellen i simulasjonen. Funksjonen `Stopwatch::millisecondsToString` tek inn ein parameter, `timeInMilliseconds`, som er ein int som angir kor mange millisekund som har gått sidan Start-knappen blei trykt på. Vi ynskjer å gjere om denne tida til eit mer leseleg format, litt som på ei stoppeklokke.

Implementer funksjonen `Stopwatch::millisecondsToString` i `simulator.cpp`.

- Konverter verdien til `timeInMilliseconds` til antal minutt, sekund, og millisekund.
- Funksjonen skal returnera ein `std::string` på formatet `'minutes.seconds.milliseconds'`.

Når oppgåve T7 er gjort vil du kunne sjå at rundetidene i kolonna *Time* oppdaterer seg under simuleringa, som vist i Figur 15.



Figur 15: Når oppgave T7 er gjort vil du kunne sjå at rundetidene i kolonnen *Time* oppdaterer seg under simuleringa.

8. (25 points) **T8: Straffetid**

For å finne det endelege resultatet ynskjer vi å kombinere tida til ein utøvar med antal treff utøvaren hadde ved å leggje til straffetid. Slik tabellen er satt opp no er tidene lagra som ein `std::string`. Vi må derfor ha moglegheit til å omgjere strengen til millisekunder for å kunne leggje til straffetid, og må gjere det motsette av det vi gjorde i førre oppgåve (T7).

Implementer funksjonen `Stopwatch::stringToMilliseconds` i `simulator.cpp`. Funksjonen tek inn ein `std::string` på formatet 'minutes:seconds:milliseconds' og skal returnera ein `int` som er tida i millisekunder.

Oppgåve T8 kan ikkje verifiserast visuelt.

9. (25 points) **T9: Kven hamnar på pallen?**

Som nemnt i førre oppgåve (T8) blir det endelege resultatet bestemt ved å kombinera tida til ein utøvar med antal treff utøvaren hadde ved å leggje til straffetid. For å finne den endelege tida til ein utøvar må vi dermed lese verdien som er lagra i kolonnene *Time* og *Hits*, konvertere begge verdiane fra strenger til tal, rekne ut og leggje til straffetid, og til slutt konvertera tida tilbake til ein streng som viser den endelege tida på eit leselig format.

Straffetida per bom er oppgitt i den utdelte koden som konstanten `PENALTY_FACTOR`:

$$\text{PENALTY_FACTOR} = \frac{\text{PENALTY}}{\text{SPEED_SCALE}}. \quad (1)$$

Klassen `Stopwatch` inneheld funksjonane `millisecondsToString` og `stringToMilliseconds` som du implementerte i oppgåve T7 og T8. Desse kan brukast til å konvertera tid fram og tilbake mellom strenger og tal. `Simulator`-klassen har ein instans av `Stopwatch` kalla `watch` lagra som ein `unique_ptr`. `Stopwatch`-klassen er definert i `simulator.h` og ser slik ut:

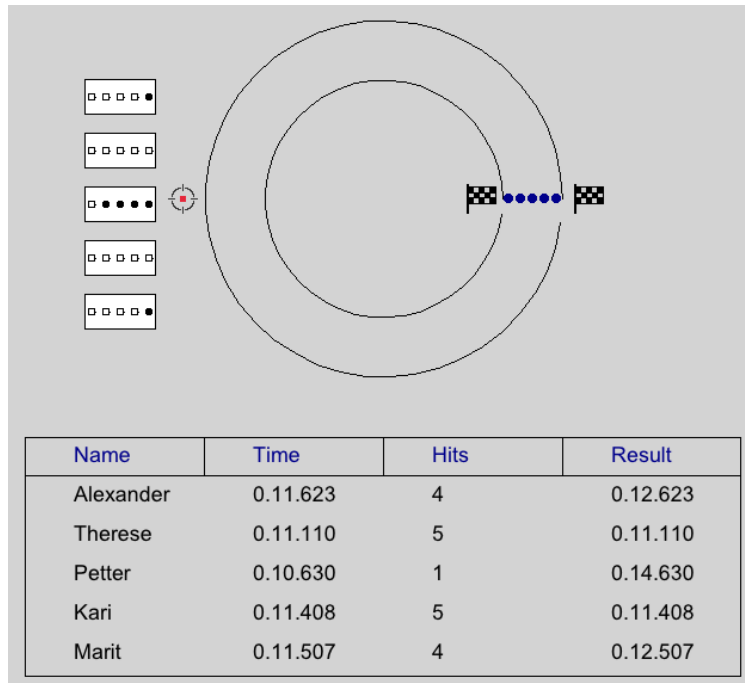
```
class Stopwatch
{
public:
    void toggleWatch();
    std::chrono::duration<int64_t, std::nano> getTime();
    string timeToString(std::chrono::duration<int64_t, std::nano> clock);
    string millisecondsToString(int time);
    int stringToMilliseconds(string time);

private:
    vector<std::chrono::time_point<std::chrono::steady_clock>> timeEvents;
};
```

Implementer funksjonen `Simulator::finaliseResult` i `simulator.cpp`. Funksjonen tek inn ei rad frå `Table::data` og skal returnera summen av rundetida og straffetida som ein `std::string` på formatet `'minutes:seconds:milliseconds'`.

- Hent ut indeksen til kolonna `Time` i rada. Dersom kolonnenamnet ikkje finst skal du returnera strengen `'Time missing'`.
- Bruk `Stopwatch`-instansen `watch` i `Simulator` til å konvertera rundetida til millisekunder.
- Hent ut indeksen til kolonna `Hits`. Dersom kolonnenamnet ikkje finst skal du returnera strengen `'Hits missing'`.
- Legg til straffetida til rundetida. Straffetida per bom finst i konstanten `PENALTY_FACTOR` (Likning (1)).
- Konverter den endelege tida (rundetida + straffetida) til ein `std::string` på formatet `'minutes:seconds:milliseconds'`.
- Returner strengen du konstruerte i punktet over.

Når oppgåve T9 er gjort vil du kunne sjå at tabellen oppdaterer *Result*-kolonna etter kvart som utøvarane kjem i mål, som vist i Figur 16.



Figur 16: Skjermbilete av simulasjonen etter at oppgåve T3 er implementert.

10. (15 points) T10: Operatoroverlasting

Vi ynskjer å forenkle prosessen med å skriva ut `TableData`-strukturer. Vi vil derfor overlaste `<<`-operatoren.

Overlast `<<`-operatoren for `TableData` i `table.h`.

- Funksjonen skal ta inn 2 parametrar, ein `std::ostream` og ein `TableData`-struktur med generell type.
- Vi ynskjer å skriva data til straumen på formatet `'TableData::columnTitle:TableData::value'`.

Oppgåve T10 kan ikkje verifiserast visuelt.

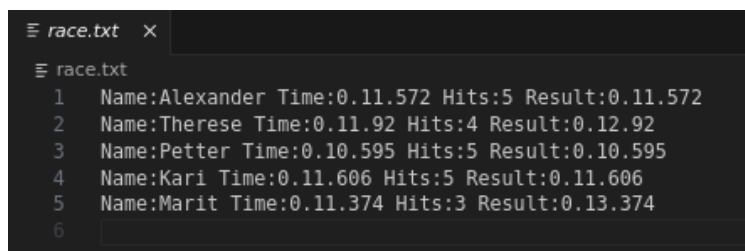
11. (20 points) T11: Lagre resultata

Vi ynskjer å lagra resultata frå simuleringa i ei fil for seinare bruk.

Implementer funksjonen `Table::storeTable` i `table.cpp`.

- Dersom `filepath` ikkje finst skal funksjonen utløyse eit passande unntak.
- Data for ein utøvar skal lagrast som ei linje i fila.
- Kolonnenamnet og kolonneverdien skal lagrast som `TableData::columnTitle:TableData::value` for kvar kolonne, med eit mellomrom mellom kolonnane. Du kan sjå filformatet i Figur 17.

Når oppgave T11 er implementert blir det lagra ei fil, `race.txt`, i hovedmappa på formatet vist i Figur 17 når simuleringa er ferdig.

A screenshot of a text editor window titled 'race.txt'. The editor shows a table with 6 rows. The first five rows contain data for different participants, and the sixth row is empty. The data is as follows:

	Name	Time	Hits	Result
1	Alexander	0.11.572	5	0.11.572
2	Therese	0.11.92	4	0.12.92
3	Petter	0.10.595	5	0.10.595
4	Kari	0.11.606	5	0.11.606
5	Marit	0.11.374	3	0.13.374
6				

Figur 17: Formatet til fila som tabellen blir lagra i.